Rocky Linux 基础操作实验任务书

先说两句

这份实验任务书是给 Linux 零基础的同学准备的。不用提前啃书，也不用先去看视频——直接上手敲命令，敲着敲着就会了。

遇到不懂的命令、看不懂的报错，打开豆包或 DeepSeek，把问题扔进去，让它当你的实时答疑。别怕问得蠢，AI 不会嫌你烦。

做完这 7 个实验，后面编译 llama.cpp、搭 AI 推理服务就没什么障碍了。

实验前确认

- [ ] VMware 装好且能正常打开
- [ ] Rocky Linux 虚拟机创建完，系统也装好了
- [ ] 能通过 `ssh` 登进你的虚拟机（Windows Terminal 或 PowerShell 里敲 `ssh` 就行）
- [ ] 拍了虚拟机快照（搞崩了能一键恢复）
- [ ] 有一个能用的 AI 工具——豆包、DeepSeek 随便哪个都行

AI 怎么用

你遇到的情况	怎么做
不确定某个命令怎么用	问："Rocky Linux 里 ls 命令有哪些常用参数？"
终端报错了	把报错信息完整复制，贴给 AI 让它分析
想搞懂一个概念	问："用通俗的话解释 Linux 里的管道"
需要操作示范	问："在 Rocky Linux 里查找大文件的命令示例"
拿不准选哪个方案	把两个方案都说一下，让 AI 对比分析

> AI 说的不一定对。动手验证，别全信。

实验一：Shell 基础，认识命令行

你要学会的

- 搞清楚 Shell、终端、命令行到底是什么关系
- 看懂命令的基本格式：`命令 [选项] [参数]`
- 会用 Tab 键自动补全（这技能省时间，一定要练）

先问 AI

问豆包或 DeepSeek："Linux 里的 Shell 是什么？和终端有什么区别？" 看完回答，用自己的话记两笔：

```
AI 说了什么（摘要）：


```

动手敲

```
# 1. 看看你在用什么 Shell
echo $SHELL

# 2. 敲 date 然后回车，看输出什么
date

# 3. 试试 Tab 补全：输入 cal 然后连按两下 Tab
# （如果提示找不到 cal，先 dnf install cal -y 装上）
```

```
# 4. 敲 history 看看刚才都执行过什么命令
#    然后按键盘 ↑（上箭头），试试调出刚才的命令
history

# 5. 用 !! 直接重跑上一条命令
!!
```

记一下输出

命令	屏幕上显示了什么
`echo $SHELL`	
`date`	
`history`	
`!!`	

想一想

Shell 命令行里，`#` 和 `$` 这两个符号分别代表什么意思？先问 AI，再写下你的理解：

```
AI 说的：


我理解的是：
```

实验二：在文件系统里到处走走，别迷路

你要学会的

- Linux 的目录长什么样（一棵倒着长的树）
- 三个导航命令：`pwd`、`cd`、`ls`
- 绝对路径和相对路径的区别——这个后面天天要用

先问 AI

问："Linux 根目录下面 /etc、/usr、/var、/home、/tmp 都是干嘛的？"

```
AI 说的（摘要）：


```

动手敲

```
# 先看看自己在哪
pwd

# ls：列出目录里的东西
ls              # 当前目录
ls /etc         # /etc 目录
ls -l           # 详细信息
ls -la          # 包括隐藏文件
ls -lh          # 文件大小显示成 KB、MB 这样人能看懂的
ls -ltr         # 按时间排，最新放最后

# cd：到处走
cd /etc
pwd             # 确认跑过来了
ls              # 看看 /etc 下面有什么

cd /            # 去根目录
ls

cd ~            # 回家（root 的话就是 /root）
pwd

cd ..           # 上一层
pwd

cd -            # 回到刚才待的地方
pwd
```

> 在 `/` 和 `/etc` 之间来回跳几次，每次用 `pwd` 确认位置，用 `ls` 看一眼。把 `/etc`、`/usr`、`/var`、`/home`、`/tmp` 都串一遍。

> 不明白某个目录是干啥的？随时问 AI："`/var/log` 目录放的是什么东西？"

做个练习

```
# 一口气执行下面这条，观察输出
cd /tmp && pwd && ls && cd /etc && pwd && ls && cd ~ && pwd
```

```
【把你看到的输出贴在这里】


```

想一想

绝对路径和相对路径到底有什么区别？什么时候该用哪种？问完 AI 用自己的话说：

```
我的理解：


```

实验三：增删改查，折腾文件和目录

你要学会的

- `mkdir`、`touch`、`cp`、`mv`、`rm`——这几个命令以后天天用
- `rm` 的危险之处（Linux 没有回收站，删了就没了）
- 怎么在目录树里复制、移动、重命名文件

先问 AI

问："Linux 里 rm -rf 命令到底危险在哪？用的时候要注意什么？"

```
AI 说的（摘要）：


```

动手敲

建目录建文件

```
mkdir ~/linux_lab
cd ~/linux_lab
pwd     # 确认一下你在正确的地方

# 一口气建多层目录（-p：父目录不存在就自动建）
mkdir -p project/src
mkdir -p project/docs
mkdir -p project/data

ls -R project    # 看看目录结构
```

```

# 建几个空文件
touch readme.txt
touch project/src/main.py
touch project/src/utils.py
touch project/docs/README.md
```

复制

```
cp readme.txt readme_backup.txt
ls

cp readme.txt project/docs/
ls project/docs/

# 复制整个目录要加 -r
cp -r project project_backup
ls -R project_backup
```

> 搞不清什么时候该加 `-r`？问 AI。

移动和改名

```
# mv 能干两件事：移动 和 重命名

mv readme_backup.txt README.txt    # 改名
ls

mv README.txt project/docs/        # 移动
ls project/docs/

mv project project_v2              # 移动目录不用加 -r
ls
```

删除——这一段认真看

```
touch temp_file.txt
ls
rm temp_file.txt        # 删单个文件，会有确认提示
ls

touch temp_file.txt
rm -f temp_file.txt     # -f 强制删，不问你
ls

mkdir temp_dir && touch temp_dir/test.txt
rm -r temp_dir          # 删目录要加 -r
ls

# 下面这条永远、永远、永远不要执行：
# rm -rf /
```

```
# 它会删光整个系统。看懂就行，别手贱。
```

练习题

在 `~/linux_lab` 底下做这几件事，每步做完用 `ls` 或 `pwd` 验证：

1. 在 `project_v2` 下面建一个 `lib` 子目录
2. 把 `project_v2/src/main.py` 复制一份到 `project_v2/lib/`
3. 把 `project_v2/src/main.py` 改名为 `app.py`
4. 把 `project_v2/docs/README.md` 挪到 `~/linux_lab/` 下面
5. 删掉整个 `project_v2` 目录

```
【把你每一步敲的命令贴在这里——方便我检查你有没有走弯路】
```

验收

```
cd ~/linux_lab && ls -la
```

```
【输出结果贴这里】
```

想一想

用 `rm` 删掉的文件能恢复吗？问 AI，然后写结论：

```
AI 说的：


我的结论：
```

实验四：看文件、写文件

你要学会的

- `cat`、`less`、`file`——看文件的三件套
- `head`、`tail`——只看头看尾
- `vim`——Linux 上最重要的文本编辑器。上手有点别扭，但绕不过去

动手敲

先看文件

```
cd ~/linux_lab

# 用 echo 和 > 往文件里塞内容
echo "第一行内容" > test.txt
echo "第二行内容" >> test.txt
echo "第三行内容" >> test.txt

cat test.txt          # 一口气全显示

# 拿 /etc/passwd 的前 50 行当练习材料
head -50 /etc/passwd > user_list.txt
wc -l user_list.txt    # 确认多少行

less user_list.txt     # 分页看（q 退出，空格翻页，/ 搜索）
```

```
head user_list.txt           # 前十行
head -3 user_list.txt        # 前三行

tail user_list.txt           # 末十行
tail -3 user_list.txt        # 末三行

# 顺便看看 file 命令
file test.txt
file /etc/passwd
file /usr/bin/bash
```

vim——认真学，这个躲不掉

先问 AI："给 Linux 新手讲讲 vim 的基本用法，重点是三种模式：普通模式、插入模式、命令模式。"

```
AI 说的（摘要）：




```

vim 速查表（把这几个记熟，够用了）：

你要干嘛	按什么	补充说明
开始写字	`i`	普通模式下按，进入插入模式
保存退出	`:wq` 回车	普通模式下输入
不保存走人	`:q!` 回车	改错了不想存
回到普通模式	`Esc`	任何时候都能按
删整行	`dd`	普通模式
撤销	`u`	普通模式
搜东西	`/关键词` 回车	`n` 跳到下一个匹配

```
vim ~/linux_lab/hello.txt

# 进 vim 后按 i，输入下面几行：
# Hello, this is my first vim file.
# I am learning Rocky Linux.
# AI tools are helping me.
#
# 然后按 Esc，再输入 :wq 保存退出

cat ~/linux_lab/hello.txt    # 确认写进去了
```

练个真的：编辑 `/etc/hosts`

```
cp /etc/hosts /etc/hosts.bak      # 先备份，搞坏了能还原
vim /etc/hosts
# 在文件末尾加一行：127.0.0.1 mytest.local
# 保存退出
cat /etc/hosts                    # 确认
```

练习题

1. 在 `~/linux_lab` 下建 `vim_practice.txt`
2. 用 vim 写至少 5 行内容，中英文随便
3. 在 vim 里练这几个操作：删一行（`dd`）、复制一行（`yy`）、粘贴（`p`）、撤销（`u`）
4. 保存后用 `cat` 确认结果

```
【大致写下你的 vim 操作过程，证明你动手了】
```

想一想

`cat /etc/passwd` 和 `file /etc/passwd` 各自输出什么？这两个命令的用途差别在哪？

```
cat /etc/passwd 输出（前3行就行）：


file /etc/passwd 输出：


我的理解：
```

实验五：用户和权限——谁能在你的系统上干什么

你要学会的

- Linux 是多用户系统，每个人各干各的
- 怎么建用户、切换用户
- `rwx` 到底什么意思
- `chmod` 和 `chown` 怎么用

先问 AI

问："用大白话解释 Linux 文件权限 rwx，还有 755 和 644 是什么意思。"

```
AI 说的（摘要）：


```

动手敲

先搞用户

```
whoami           # 我是谁
id               # 我的详细信息

useradd student
passwd student       # 设密码，输的时候不显示，正常

cat /etc/passwd | tail -10     # 看看系统里有哪些用户

su - student         # 切到 student
whoami               # 确认身份变了
pwd                  # 注意：普通用户家目录在 /home/student
exit                 # 退回去
```

> 切到 student 之后，命令行提示符从 `#` 变成了 `$`。`#` 是管理员，`$` 是普通用户。这是个重要的区分——以后看到 `$` 提示符就别乱改系统文件了。

然后权限

```
cd ~/linux_lab

touch permission_test.txt
ls -l permission_test.txt

# -rw-r--r--  这串东西怎么读：
# 第一个 -  文件类型（-是文件，d是目录，l是链接）
# rw-       文件主人的权限（读写，不能执行）
# r--       同组用户的权限（只读）
# r--       其他人的权限（只读）

chmod 755 permission_test.txt    # rwx r-x r-x
ls -l permission_test.txt

chmod 644 permission_test.txt    # rw- r-- r--
ls -l permission_test.txt

# 建个脚本试试执行权限
echo 'echo "hello from script"' > myscript.sh
cat myscript.sh
./myscript.sh              # 报错 Permission denied——没执行权限
chmod +x myscript.sh       # 加上执行权限
./myscript.sh              # 这次能跑了
```

练习

```
# 1. student 用户自己建个文件
su - student
cd /tmp
touch student_file.txt
ls -l student_file.txt     # 谁建的？看所有者
exit

# 2. root 把文件抢过来
cd /tmp
chown root:root student_file.txt
ls -l student_file.txt     # 所有者变成 root 了

# 3. student 回来试试删这个文件
su - student
rm /tmp/student_file.txt   # 能删掉吗？想想为什么。
exit
```

验收

```
grep student /etc/passwd
```

```
【输出贴这里】

```

想一想

如果一个目录权限是 `rwx--x--x`，别人能 `ls` 这个目录吗？能 `cd` 进去吗？

先自己想，再动手验证，最后问 AI 确认。

```
我自己想的：


实际试一下的结果：


AI 确认：
```

实验六：装软件、卸软件

你要学会的

- `dnf` 怎么用——搜索、安装、卸载、更新
- EPEL 仓库是什么、怎么开
- 开发工具组（gcc、make 这些）怎么一口气装完

动手敲

dnf 基本操作

```
dnf search vim           # 搜
dnf info vim             # 看详情

dnf install htop -y      # 装
which htop
htop --version

dnf list installed | tail -20    # 看看都装了些什么

dnf remove htop -y       # 卸
which htop               # 没了

# 知道命令但不知道它属于哪个包？反过来查：
dnf provides javac
dnf provides */nginx
```

开 EPEL 仓库

EPEL 是第三方软件仓库，很多东西官方源里没有，得开这个才有。

```
dnf install epel-release -y
/usr/bin/crb enable

dnf repolist            # 看看现在有多少包能用
```

装开发工具——后面编译 llama.cpp 全靠它们

```
dnf group install "Development Tools" -y

# 验证一下
gcc --version
g++ --version
make --version

# git、wget、curl 这些顺手也装了
dnf install git wget curl vim htop tmux -y
```

> 这一步装完，编译代码的工具链基本齐了。

练习题

1. 用 `dnf search` 搜 `python` 相关的包，找 3 个出来
2. 查看 `git` 包的详细信息
3. 查查哪个包提供了 `gcc` 命令
4. 列出已安装的包中名字带 `dev` 的

```
【命令和输出贴这里】
```

想一想

用 `dnf` 装软件和自己下载源码编译，各有什么好处？什么时候必须自己编译？

```
我的回答（先问 AI 再总结）：
```

实验七：进程和系统监控——看看系统在忙什么

你要学会的

- 什么叫进程
- `ps`、`top`（或 `htop`）怎么看进程
- 怎么用 `kill` 杀掉不听话的进程
- 怎么看 CPU 和内存用了多少

动手敲

```
ps                    # 当前终端跑的进程
echo $$               # Shell 自己的 PID

ps aux | head -20     # 系统所有进程，看前 20 个
ps -ef | head -20     # 另一种风格，输出差不多

ps aux | grep ssh     # 只找 ssh 相关进程

top                   # 实时看（q 退出）
htop                  # 比 top 好看，前提是你装了
```

```
# 让一个命令在后台跑
sleep 300 &
echo "PID 是 $!"

ps -p $!              # 看看这个进程还在不在

kill $!               # 干掉它
# kill -9 $!          # 如果普通 kill 没反应，用这个强制杀——但别养成习惯

# 看看系统资源
free -h               # 内存还剩多少
df -h                 # 磁盘还剩多少
uptime                # 开机多久了，负载怎么样
```

练习

开两个 SSH 窗口同时登进系统：

- **窗口一：** `sleep 600 &`（让它在后台跑 10 分钟）
- **窗口二：** `ps aux | grep sleep` 找到那个进程，然后 `kill` 掉
- **回到窗口一：** `jobs` 看看进程还在不在

```
【记录你的操作过程】
```

想一想

`kill` 和 `kill -9` 到底有什么区别？什么时候打死也不能用 `kill -9`？

```
AI 说的：
```